String and Array
Chapter -9

String Primitive Instructions

String is s series of data byte or word available in memory at consecutive locations.

- It is either referred as byte string or word string.

Their memory is always allocated in a sequential order.

Instructions used to manipulate strings are called string manipulation instructions.

Instruction	Description
MOVSB, MOVSW, MOVSD	Move string data: Copy data from memory addressed by ESI to memory addressed by EDI.
CMPSB, CMPSW, CMPSD	Compare strings: Compare the contents of two memory locations addressed by ESI and EDI.
SCASB, SCASW, SCASD	Scan string: Compare the accumulator (AL, AX, or EAX) to the contents of memory addressed by EDI.
STOSB, STOSW, STOSD	Store string data: Store the accumulator contents into memory addressed by EDI.
LODSB, LODSW, LODSD	Load accumulator from string: Load memory addressed by ESI into the accumulator.

String Primitive Instructions

- Although they are called *string primitives*, they are not limited to character arrays.
- Each instruction implicitly uses **ESI**, **EDI**, or both registers to address memory.
- String primitives execute efficiently because they automatically repeat and increment array indexes.

MOVSB, MOVSW, AND MOVSD

- The MOVSB, MOVSW, and MOVSD instructions copy data from the memory location pointed to by ESI to the memory location pointed to by EDI.

```
.data
    source DWORD 0FFFFFFFFh
    target DWORD ?
.code
    mov esi,OFFSET source
    mov edi,OFFSET target
    movsd
```

MOVSB	Move (copy) bytes
MOVSW	Move (copy) words
MOVSD	Move (copy) doublewords

- Depending upon **Direction Flag, ESI** and **EDI** are automatically incremented or decremented.

Instruction	Value Added or Subtracted from ESI and EDI
MOVSB	1
MOVSW	2
MOVSD	4

USING A REPEAT PREFIX

- By itself, a string primitive instruction processes only a single memory value or pair of values.
- If you add a *repeat prefix*, the instruction repeats, using ECX as a counter.
  - The repeat prefix permits you to process an entire array using a single instruction.

REP	Repeat while ECX > 0
REPZ, REPE	Repeat while the Zero flag is set and ECX > 0
REPNZ, REPNE	Repeat while the Zero flag is clear and ECX > 0

EXAMPLE: COPY A STRING

```
cld                          ; clear direction flag
mov esi,OFFSET string1       ; ESI points to source
mov edi,OFFSET string2       ; EDI points to target
mov ecx,10                   ; set counter to 10
rep movsb                    ; move 10 bytes
```

- If **ECX > 0**, ECX is decremented and the instruction repeats; else the control is passed to the next line in the program.
- ESI and EDI are automatically incremented when MOVSB repeats.

Direction Flag

- String primitive instructions increment or decrement ESI and EDI based on the state of the Direction flag.

Value of the Direction Flag	Effect on ESI and EDI	Address Sequence
Clear	Incremented	Low-high
Set	Decremented	High-low

- The Direction flag can be explicitly modified using the CLD and STD instructions:

```
CLD    ; clear Direction flag (forward direction)
STD    ; set Direction flag (reverse direction)
```

EXAMPLE: COPY DOUBLEWORD ARRAY

```
.data
        source DWORD 20 DUP(0FFFFFFFFh)
        target DWORD 20 DUP(?)
.code
        cld
        mov ecx,LENGTHOF source
        mov esi,OFFSET source
        mov edi,OFFSET target
        rep movsd
```

YOUR TURN . . .

Use MOVSD to delete the first element of the following doubleword array. All subsequent array values must be moved one position forward toward the beginning of the array:

```
.data
     array DWORD 1,1,2,3,4,5,6,7,8,9,10

.code
   cld
   mov ecx,(LENGTHOF array) - 1
   mov esi,OFFSET array+4
   mov edi,OFFSET array
   rep movsd
```

CMPSB, CMPSW, AND CMPSD

- The **CMPSB, CMPSW,** and **CMPSD** instructions each compare a memory operand pointed to by ESI to a memory operand pointed to by EDI:

CMPSB	Compare bytes
CMPSW	Compare words
CMPSD	Compare doublewords

- Repeat (**rep, repe, repz, repne, repnz**)can be used with CMPSB, CMPSW, and CMPSD.
- The Direction flag determines the incrementing or decrementing of ESI and EDI.

COMPARING A PAIR OF DOUBLEWORDS

```
.data
   source DWORD 1234h
   target DWORD 5678h

.code
   mov esi,OFFSET source
   mov edi,OFFSET target
   cmpsd
   ja L1                ; jump if source > target
   jmp L2               ; jump if source <= target
```

COMPARING MULTIPLE DOUBLE DOUBLEWORDS

- Clear the Direction flag (forward direction), initialize ECX as a counter, and use a repeat prefix with CMPSD

```
.data
        source DWORD count DUP(?)
        target DWORD count DUP(?)
.code
        mov ecx,LENGTHOF source
        mov esi,OFFSET source
        mov edi,OFFSET target
        cld                     ; direction = forward
        repe cmpsd              ; repeat while equal
```

EXAMPLE: COMPARING TWO STRINGS (1 OF 3)

```
.data
   source BYTE "MARTIN  "
   dest   BYTE "MARTINEZ"
   str1 BYTE "Source is smaller",0dh,0ah,0
   str2 BYTE "Source is not smaller",0dh,0ah,0
```

Screen output:

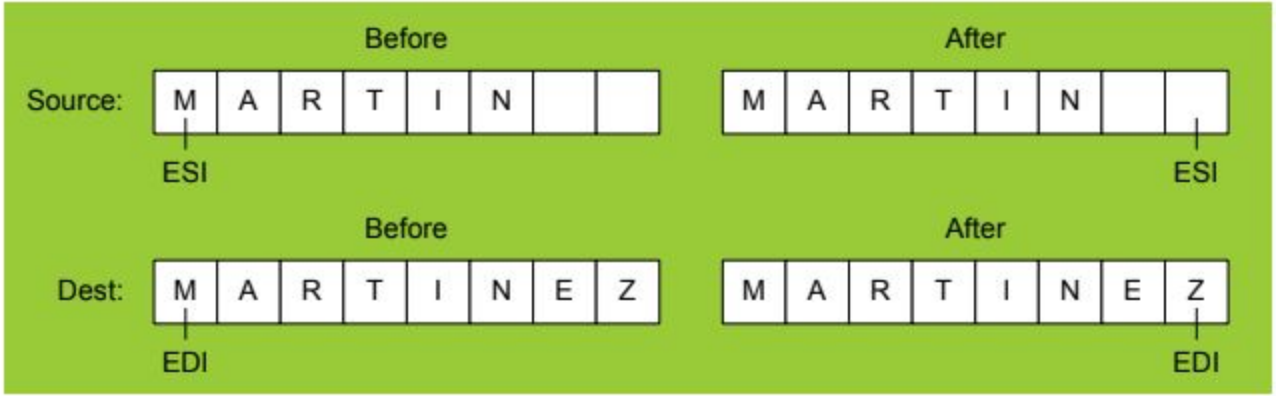
```
Source is smaller
```

EXAMPLE: COMPARING TWO STRINGS (2 OF 3)

```
.code
main PROC
   cld                    ; direction = forward
   mov  esi,OFFSET source
   mov  edi,OFFSET dest
   mov  ecx,LENGTHOF source
   repe cmpsb
   jb   source_smaller
   mov  edx,OFFSET str2 ; "source is not smaller"
   jmp  done
source_smaller:
   mov  edx,OFFSET str1 ; "source is smaller"
done:
   call WriteString
   exit
main ENDP
```

EXAMPLE: COMPARING TWO STRINGS (3 OF 3)

The following diagram shows the final values of ESI and EDI after comparing the strings:



SCASB, SCASW, AND SCASD

- The **SCASB** instruction compares a value in AL to a byte addressed by EDI.
- **SCASW** instruction compares a value in AX to a word addressed by EDI.
- **SCASD** instruction compares a value in EAX to a doubleword addressed by EDI.
- The Direction flag determines the incrementing or decrementing of EDI.
- The instructions are useful when looking for a single value in a string or array.

EXAMPLE: SCAN FOR A MATCHING CHARACTER

```
.data
alpha BYTE "ABCDEFGH",0
.code
mov edi,OFFSET alpha      ; EDI points to the string
mov al,'F'                ; search for the letter F
mov ecx,LENGTHOF alpha    ; set the search count
Cld                       ; direction = forward
repne scasb               ; repeat while not equal
jnz quit                  ; quit if letter not found
dec edi                   ; found: back up EDI
```

STOSB, STOSW, AND STOSD

- The **STOSB**, **STOSW**, and **STOSD** instructions store the contents of AL/AX/EAX, respectively, in memory at the offset pointed to by EDI.
- EDI is incremented or decremented based on the state of the Direction flag.
- When used with the REP prefix, these instructions are useful for filling all elements of a string or array with a single value

EXAMPLE: FILL AN ARRAY WITH 0FFH

```
.data
Count = 100
string1 BYTE Count DUP(?)
.code
mov al,0FFh              ; value to be stored
mov edi,OFFSET string1   ; ES:DI points to target
mov ecx,Count            ; character count
cld                      ; direction = forward
rep stosb                ; fill with contents of AL
```

LODSB, LODSW, AND LODSD

- The **LODSB, LODSW,** and **LODSD** instructions load a byte/word/doubleword from memory at ESI into AL/AX/EAX, respectively.
- ESI is incremented or decremented based on the state of the Direction flag.
- The REP prefix is rarely used with LODS because each new value loaded into the accumulator overwrites its previous contents.
  - Instead, LODS is used to load a single value.

EXAMPLE: ARRAY MULTIPLICATION EXAMPLE

```
.data
  array DWORD 1,2,3,4,5,6,7,8,9,10  ; test data
  multiplier DWORD 10               ; test data
.code
 main PROC
  cld
  mov esi,OFFSET array
  mov edi,esi
  mov ecx,LENGTHOF array
  L1: lodsd                         ; load [ESI] into EAX
  mul multiplier                    ; multiply by a value
  stosd                             ; store EAX into [EDI]
  loop L1
```

Your turn . . .

- Write a program that converts each unpacked binary-coded decimal byte belonging to an array into an ASCII decimal byte and copies it to a new array.

```
.data
array BYTE 1,2,3,4,5,6,7,8,9
dest  BYTE (LENGTHOF array) DUP(?)
```

Your turn . . .

- Write a program that converts each unpacked binary-coded decimal byte belonging to an array into an ASCII decimal byte and copies it to a new array.

```
.data
array BYTE 1,2,3,4,5,6,7,8,9
dest  BYTE (LENGTHOF array) DUP(?)
```

```
    mov esi,OFFSET array
    mov edi,OFFSET dest
    mov ecx,LENGTHOF array
    cld
L1: lodsb  ; load into AL
    or al,30h ; convert to ASCII
    stosb  ; store into memory
    loop L1
```

Selected String Procedures

The following string procedures may be found in the Irvine32 and Irvine16 libraries:

- Str_compare Procedure
- Str_length Procedure
- Str_copy Procedure
- Str_trim Procedure
- Str_ucase Procedure

Str_compare Procedure

- Compares *string1* to *string2*, setting the Carry and Zero flags accordingly
- Prototype:

```
Str_compare PROTO,
  string1:PTR BYTE,    ; pointer to string
  string2:PTR BYTE     ; pointer to string
```

Relation	Carry Flag	Zero Flag	Branch if True
string1 < string2	1	0	JB
string1 == string2	0	1	JE
string1 > string2	0	0	JA

Str_compare Source Code

```
Str_compare PROC USES eax edx esi edi,
    string1:PTR BYTE, string2:PTR BYTE
     mov esi,string1
     mov edi,string2
L1:  mov  al,[esi]
     mov  dl,[edi]
     cmp  al,0              ; end of string1?
     jne  L2                ; no
     cmp  dl,0              ; yes: end of string2?
     jne  L2                ; no
     jmp  L3                ; yes, exit with ZF = 1
L2:  inc  esi               ; point to next
     inc  edi
     cmp  al,dl             ; chars equal?
     je   L1                ; yes: continue loop
L3:  ret
Str_compare ENDP
```

Str_length Procedure

- Calculates the length of a null-terminated string and returns the length in the EAX register.
- Prototype:

```
Str_length PROTO,
  pString:PTR BYTE     ; pointer to string
```

Example:

```
.data
myString BYTE "abcdefg",0
.code
    INVOKE Str_length,
      ADDR myString
; EAX = 7
```

Str_length Source Code

```
Str_length PROC USES edi,
    pString:PTR BYTE     ; pointer to string

    mov edi,pString
    mov eax,0            ; character count
L1:
    cmp byte ptr [edi],0        ; end of string?
    je  L2              ; yes: quit
    inc edi             ; no: point to next
    inc eax             ; add 1 to count
    jmp L1
L2: ret
Str_length ENDP
```

Str_copy Procedure

- Copies a null-terminated string from a source location to a target location.
- Prototype:

```
Str_copy PROTO,
  source:PTR BYTE, ; pointer to string
  target:PTR BYTE  ; pointer to string
```

See the CopyStr.asm program for a working example.

Str_copy Source Code

```
Str_copy PROC USES eax ecx esi edi,
    source:PTR BYTE,          ; source string
    target:PTR BYTE       ; target string

    INVOKE Str_length,source    ; EAX = length source
    mov ecx,eax             ; REP count
    inc ecx                 ; add 1 for null byte
    mov esi,source
    mov edi,target
    cld                         ; direction = up
    rep movsb               ; copy the string
    ret
Str_copy ENDP
```

Str_trim Procedure

- The Str_trim procedure removes all occurrences of a selected trailing character from a null-terminated string.
- Prototype:

```
Str_trim PROTO,
  pString:PTR BYTE,    ; points to string
  char:BYTE            ; char to remove
```

Example:

```
.data
myString BYTE "Hello###",0
.code
   INVOKE Str_trim, ADDR myString, '#'

myString = "Hello"
```

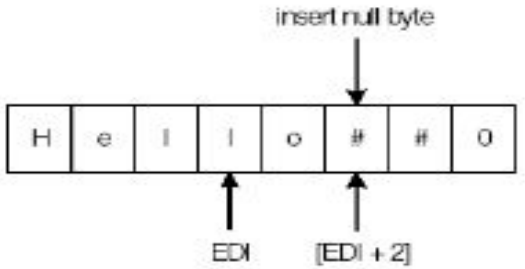
Str_trim Procedure

- Str_trim checks a number of possible cases (shown here with # as the trailing character):
  - The string is empty.
  - The string contains other characters followed by one or more trailing characters, as in "Hello##".
  - The string contains only one character, the trailing character, as in "#"
  - The string contains no trailing character, as in "Hello" or "H".
  - The string contains one or more trailing characters followed by one or more nontrailing characters, as in "#H" or "###Hello".

Testing the Str_trim Procedure

String Definition	EDI, When SCASB Stops	Zero Flag	ECX	Position to Store the Null
`str BYTE "Hello##",0`	str + 3	0	> 0	[edi + 2]
`str BYTE "#",0`	str – 1	1	0	[edi + 1]
`str BYTE "Hello",0`	str + 3	0	> 0	[edi + 2]
`str BYTE "H",0`	str – 1	0	0	[edi + 2]
`str BYTE "#H",0`	str + 0	0	> 0	[edi + 2]

Using the first definition in the table, position of EDI when SCASB stops:



Str_trim Source Code

```
Str_trim PROC USES eax ecx edi,
    pString:PTR BYTE,    ; points to string
    char:BYTE ; char to remove
    mov  edi,pString
    INVOKE Str_length,edi   ; returns length in EAX
    cmp  eax,0    ; zero-length string?
    je   L2    ; yes: exit
    mov  ecx,eax  ; no: counter = string length
    dec  eax
    add  edi,eax  ; EDI points to last char
    mov  al,char  ; char to trim
    std ; direction = reverse
    repe scasb    ; skip past trim character
    jne  L1    ; removed first character?
    dec  edi   ; adjust EDI: ZF=1 && ECX=0
L1: mov  BYTE PTR [edi+2],0 ; insert null byte
L2: ret
Str_trim ENDP
```

Str_ucase Procedure

- The Str_ucase procedure converts a string to all uppercase characters. It returns no value.
- Prototype:

```
Str_ucase PROTO,
  pString:PTR BYTE     ; pointer to string
```

Example:

```
.data
myString BYTE "Hello",0
.code
    INVOKE Str_ucase,
      ADDR myString
```

Str_ucase Source Code

```
Str_ucase PROC USES eax esi,
    pString:PTR BYTE
    mov esi,pString

L1: mov al,[esi]      ; get char
    cmp al,0       ; end of string?
    je  L3     ; yes: quit
    cmp al,'a'         ; below "a"?
    jb  L2
    cmp al,'z'         ; above "z"?
    ja  L2
    and BYTE PTR [esi],11011111b   ; convert the char

L2: inc esi         ; next char
    jmp L1

L3: ret
Str_ucase ENDP
```